



COSMOPREDICT DEPLOYMENT ON AWS EC2

Project Information

Project Name

CosmoPredict – AI Powered Space Weather & Asteroid Monitoring Platform

Live Website

 <https://cosmopredict.space>

Technology Stack

- Frontend: React + Vite + TypeScript + Tailwind CSS
- Backend: Django + Django REST Framework
- AI/ML: TensorFlow + Scikit-learn
- Database: SQLite
- Web Server: Nginx
- SSL: Let's Encrypt (Win-ACME)
- Hosting: AWS EC2 Windows Server

1. Introduction

CosmoPredict is a B.Tech Final Year Major Project designed for real-time space weather monitoring, asteroid risk analysis, and AI-based prediction systems.

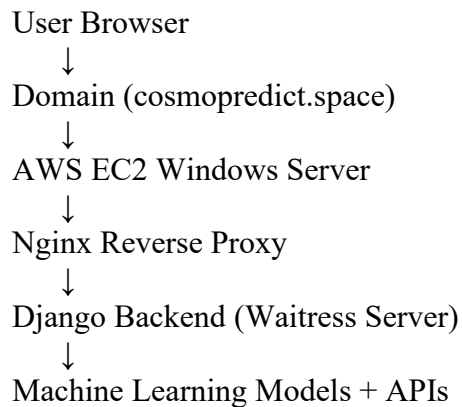
The platform integrates:

- NOAA Space Weather APIs
- NASA Asteroid APIs
- Machine Learning models
- Real-time alert systems
- Interactive dashboard
- AI-powered predictions

The complete application has been deployed on an AWS EC2 Windows instance with a custom domain and SSL security.

2. AWS Architecture Overview

Deployment Architecture



3. AWS EC2 Instance Setup

Step 1: Launch EC2 Instance

AWS Services Used

- Amazon EC2
- Security Groups
- Elastic IP
- Route 53 / Hostinger DNS

Instance Configuration

Setting	Value
Instance Type	t2.medium / t3.medium
Operating System	Windows Server
Storage	30 GB
Public IP	Enabled
Security Group	Custom

Step 2: Configure Security Group

Inbound Rules

Type	Protocol	Port	Source
HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0
Custom TCP	TCP	8000	0.0.0.0/0
RDP	TCP	3389	My IP

Screenshot Placeholder

Insert Screenshot: AWS EC2 Security Group Rules

4. Domain Configuration

Domain Provider

Hostinger

Domain Name

 cosmopredict.space

DNS Records

A Record

Type	Name	Content
A	@	EC2 Public IP

CNAME Record

Type	Name	Content
CNAME	www	cosmopredict.space

Screenshot Placeholder

Insert Screenshot: Hostinger DNS Configuration

5. Backend Deployment (Django)

Step 1: Clone GitHub Repository

`git clone https://github.com/dfordivyansh/Cosmo-Predict.git`

Step 2: Create Virtual Environment

```
python -m venv venv
```

Activate Environment

```
venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

6. Django Production Configuration

settings.py Configuration

Important Production Settings

```
DEBUG = False
```

```
ALLOWED_HOSTS = ['*']
```

```
STATIC_URL = '/static/'
```

```
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

```
STATICFILES_DIRS = [BASE_DIR / 'static']
```

Static File Collection

```
python manage.py collectstatic
```

Database Migration

```
python manage.py migrate
```

Create Django Superuser

```
python manage.py createsuperuser
```

7. Running Django with Waitress

Install Waitress

```
pip install waitress
```

Start Production Server

```
waitress-serve --host=0.0.0.0 --port=8000 config.wsgi:application
```

Backend URL

```
http://127.0.0.1:8000
```

8. Frontend Deployment (React + Vite)

Install Node Modules

`npm install`

Fix Memory Issue During Build

Build Command

`set NODE_OPTIONS=--max-old-space-size=4096 && npm run build`

Build Output

After successful build:

`/dist`

folder is generated.

Deploy React Build

Copy all contents of:

`/dist`

into:

`C:\nginx\html`

9. Nginx Installation & Configuration

Install Nginx on Windows

Download Nginx:

<https://nginx.org/en/download.html>

Extract into:

`C:\nginx`

Final Nginx Configuration

```
worker_processes 1;
```

```
events {  
    worker_connections 1024;  
}
```

```
http {
```

```

include    mime.types;
default_type application/octet-stream;

sendfile    on;
keepalive_timeout 65;

server {
    listen 80;
    server_name cosmopredict.space www.cosmopredict.space;

    location ^~ /.well-known/acme-challenge/ {
        root C:/nginx/html;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name cosmopredict.space www.cosmopredict.space;

    ssl_certificate    C:/nginx/ssl/cosmopredict.space-chain.pem;
    ssl_certificate_key C:/nginx/ssl/cosmopredict.space-key.pem;

    ssl_protocols TLSv1.2 TLSv1.3;

    location /static/ {
        alias C:/Users/Administrator/Desktop/Cosmo-Predict/cosmo-backend/config/staticfiles/;
    }

    location /admin/ {
        proxy_pass http://127.0.0.1:8000;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
    }

    location / {
        root C:/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}

```

```
}  
}
```

Start Nginx

start nginx

Reload Nginx

nginx -s reload

Test Nginx

nginx -t

10. SSL Certificate Setup (Let's Encrypt)

SSL Tool Used

Win-ACME

Download

<https://www.win-acme.com/>

SSL Generation Process

Certificate Type

- RSA Key
- PEM Encoded Files

SSL Files Generated

cosmopredict.space-key.pem

cosmopredict.space-chain.pem

Stored in:

C:\nginx\ssl

HTTPS Enabled

Final Secure URL:

 <https://cosmopredict.space>

11. Firewall Configuration

Open Windows Firewall Ports

Allow HTTP

New-NetFirewallRule -DisplayName "HTTP 80" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow

Allow HTTPS

New-NetFirewallRule -DisplayName "HTTPS 443" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow

Allow Django

New-NetFirewallRule -DisplayName "Django 8000" -Direction Inbound -Protocol TCP -LocalPort 8000 -Action Allow

12. API Integration

External APIs Used

NOAA Space Weather API

Used for:

- KP Index
- Solar Storm Data
- Space Weather Alerts

NASA APIs

Used for:

- Near Earth Objects
- Asteroid Monitoring
- Hazard Prediction

13. Machine Learning Features

AI Models Used

- Random Forest Classifier

- TensorFlow Models
- LSTM Architecture

Prediction Features

- Asteroid Hazard Prediction
- Space Weather Forecasting
- Solar Activity Analysis

14. Project Features

Main Features

Frontend Features

- Responsive UI
- Real-time Dashboard
- Graphical Data Visualization
- Interactive Charts
- Alert System

Backend Features

- REST APIs
- Real-time Data Fetching
- Scheduler Automation
- ML Integration
- Admin Dashboard

Security Features

- HTTPS SSL Encryption
- Reverse Proxy Protection
- Secure API Routing
- Production Server Deployment

15. Admin Panel

Django Admin URL

<https://cosmopredict.space/admin>

Features

- Data Monitoring
- Alert Management

- API Logs
- Space Weather Records
- ML Predictions

16. Challenges Faced During Deployment

Major Issues Resolved

1. GitHub Large File Error

Problem:

- TensorFlow DLL files exceeded GitHub size limit.

Solution:

- Added virtual environment to .gitignore
- Reinitialized Git repository

2. Static Files Not Loading

Problem:

- CSS not applied in production.

Solution:

- Configured STATIC_ROOT
- Used collectstatic
- Configured Nginx static path

3. Nginx Reverse Proxy Issues

Problem:

- API endpoints not accessible.

Solution:

- Proper proxy_pass configuration
- Fixed path routing

4. SSL Certificate Issues

Problem:

- HTTPS timeout and invalid SSL path.

Solution:

- Configured PEM certificates correctly
- Enabled HTTPS on port 443

5. React Build Memory Error

Problem:

- Heap Out Of Memory during Vite build.

Solution:

set NODE_OPTIONS=--max-old-space-size=4096 && npm run build

17. Final Deployment URLs

Main Website

 <https://cosmopredict.space>

API Dashboard

<https://cosmopredict.space/api/dashboard/>

<https://cosmopredict.space/api/history/>

<https://cosmopredict.space/api/all-alerts/>

<https://cosmopredict.space/api/range/>

<https://cosmopredict.space/api/analytics/>

<https://cosmopredict.space/api/prediction/>

<https://cosmopredict.space/api/alert-stats/>

<https://cosmopredict.space/api/nasa/>

Django Admin

<https://cosmopredict.space/admin/>

18. Future Enhancements

Planned Features

- Docker Deployment
- Kubernetes Scaling
- PostgreSQL Migration

- Redis Caching
- WebSocket Real-time Alerts
- Cloudflare CDN Integration
- CI/CD Automation

19. Conclusion

The CosmoPredict project was successfully deployed on AWS EC2 Windows Server using Django, React, Nginx, and SSL encryption.

The deployment process included:

- AWS EC2 server configuration
- Domain mapping
- HTTPS security setup
- Reverse proxy integration
- Static file optimization
- AI model deployment
- Real-time API integration

The final deployment achieved:

- ✓ Secure HTTPS Hosting
- ✓ Production-Ready Architecture
- ✓ Real-time AI Predictions
- ✓ Responsive Frontend
- ✓ Scalable Backend APIs

20. References

Official Documentation

- Django Documentation
- React Documentation
- Nginx Documentation
- AWS EC2 Documentation
- Let's Encrypt Documentation
- NOAA API Documentation
- NASA API Documentation

Author Information

Developed By

Divyansh Srivastava , Krishna Kant Sharma , Ajay Kumar , Shantanu Shahi

B.Tech Final Year Major Project

Project Name

CosmoPredict

Live Website

 <https://cosmopredict.space>